

Simulador de Escalonamento de Processos

Elder Rayan Oliveira Silva

UFCA - Universidade Federal do Cariri
@eldrayan

Espedito Ramom Mascena Ricarto

UFCA - Universidade Federal do Cariri
@RamomRicarto

Manoel Junio Duarte da Silva

UFCA - Universidade Federal do Cariri
@Junio404

Pedro Yan Alcantara Palácio

UFCA - Universidade Federal do Cariri
@pedropalacio

Sabrina Alencar Soares

UFCA - Universidade Federal do Cariri
@sabrinaalencar

Samuel Wagner Tiburi Silveira

UFCA - Universidade Federal do Cariri
@samsilveira

Sebastião Sousa Soares

UFCA - Universidade Federal do Cariri
@SebastiaoSoares

Resumo—Este artigo apresenta o desenvolvimento e a avaliação estatística de um simulador de escalonamento de processos para sistemas operacionais. Foram comparadas três políticas clássicas (FCFS, Round Robin e Prioridade) e uma abordagem customizada denominada Prioridade Dinâmica Baseada em Histórico (PDBH). Através da execução de 1.600 cargas de trabalho distribuídas em quatro cenários (Equilibrado, I/O-Bound, CPU-Bound e Desbalanceado), avaliou-se o *trade-off* entre o Tempo de Retorno (*Turnaround*) e a Equidade (Índice de Jain). Os resultados demonstram a severidade da inanição em políticas de prioridade estática, os altos custos de troca de contexto do Round Robin e a ineficácia de pontuações dinâmicas de histórico em arquiteturas estritamente não preemptivas, onde o PDBH apresentou equivalência estatística ao FCFS.

Index Terms—Sistemas Operacionais, Escalonamento de Processos, Simulação.

I. INTRODUÇÃO

"O escalonamento de processos é uma das funções mais críticas de um Sistema Operacional, responsável por multiplexar o uso do processador (CPU) entre diversas tarefas concorrentes. A escolha da política de escalonamento afeta diretamente o desempenho percebido pelo usuário e a eficiência global do sistema, exigindo um equilíbrio constante entre métricas conflitantes, como a minimização do tempo de espera e a garantia de justiça (equidade) na alocação de recursos.

Políticas clássicas apresentam comportamentos teóricos bem estabelecidos: abordagens baseadas em ordem de chegada (FCFS) são simples, mas vulneráveis ao efeito comboio; o fatiamento de tempo (*Round Robin*) promove justiça à custa de frequentes trocas de contexto; e algoritmos baseados em prioridade estática otimizam a execução de tarefas críticas, mas introduzem o risco crônico de inanição (*starvation*) para processos menos favorecidos.

Para analisar essas dinâmicas de forma empírica e reproduzível, este trabalho apresenta o desenvolvimento de um simulador de eventos discretos. Além de implementar as políticas clássicas, o projeto propõe e avalia a Prioridade Dinâmica Baseada em Histórico (PDBH), um algoritmo não preemptivo que visa mitigar a inanição através da aplicação de bônus por

tempo de espera e penalidades por consumo acumulado de CPU.

Através de um rigoroso protocolo experimental com 1.600 execuções, este artigo avalia o desempenho estrutural e estatístico desses algoritmos sob quatro cenários distintos de carga de trabalho. O objetivo principal é quantificar os *trade-offs* sistêmicos e investigar a eficácia mecânica da pontuação dinâmica (PDBH) em um ambiente não preemptivo.

II. MODELO DA SIMULAÇÃO

Esta seção descreve o modelo arquitetural adotado pelo motor de simulação para representar o comportamento de um sistema operacional monoprocesso.

A. Tempo discreto e eventos

A simulação é baseada em eventos e controlada por uma unidade de tempo discreta denominada *tick*. O motor avança no tempo processando as transições de estado dos processos.

B. Processos e rajadas

Cada processo é modelado como uma sequência alternada de rajadas de execução na CPU e requisições de Entrada/Saída (E/S). Os processos chegam ao sistema em instantes de tempo predeterminados e são inseridos na fila de prontos.

C. Modelo de E/S

Quando um processo solicita uma operação de E/S, ele abandona a CPU e passa para o estado de bloqueio. Ao concluir o tempo necessário para a operação, o processo retorna à fila de prontos, recebendo um novo instante de entrada (*ready_since*) para fins de desempate e cálculo de espera.

D. Troca de contexto

A troca de contexto impõe um custo fixo ao sistema. Quando a CPU é repassada para um processo com Identificador de Processo (PID) diferente do que estava em execução anteriormente, adiciona-se o custo de 1 *tick* ao tempo total. O primeiro despacho da simulação não acarreta esse custo.

E. Informações observáveis pelos escalonadores

Para garantir a ausência de conhecimento futuro, os escalonadores tomam decisões baseadas exclusivamente no estado presente e passado. Nenhuma política tem acesso à duração de rajadas futuras, tempo restante de execução ou informações sobre processos que ainda não chegaram ao sistema.

III. ALGORITMOS

Esta seção detalha os algoritmos de escalonamento implementados.

A. FCFS

O FCFS (*First-Come, First-Served*) foi implementado como uma política não preemptiva. A fila de prontos é ordenada pelo par (*ready_since*, *PID*): seleciona-se primeiro o processo que está há mais tempo continuamente pronto e, quando dois processos ficam prontos no mesmo instante, o menor *PID* determina o desempate. Ao retornar de E/S, o processo recebe um novo *ready_since* correspondente ao instante da conclusão da operação e volta a competir com os demais processos prontos.

A política é consultada apenas quando a CPU está livre. Portanto, uma chegada ou conclusão de E/S não interrompe o processo em execução. Para manter a comparação reproduzível e impedir conhecimento futuro, o FCFS recebe somente uma visão com *PID*, prioridade estática, chegada já observada e instante atual de entrada na fila; a sequência de rajadas, a CPU restante e as chegadas futuras permanecem sob responsabilidade exclusiva do motor da simulação. Processos que ficam prontos simultaneamente são entregues à interface em ordem crescente de *PID*, tornando o resultado independente da ordem de alocação ou de percurso das estruturas internas.

B. Round Robin

O Round Robin (RR) usa uma fila FIFO e quantum padrão de 4 ticks, que pode ser configurado por execução e deve ser um inteiro positivo. A cada despacho o contador do quantum é reiniciado. Se a rajada de CPU termina antes ou exatamente no mesmo tick em que o quantum se esgota, prevalece a conclusão natural: o processo termina ou inicia E/S sem ser registrado como preemptado, e qualquer fração restante do quantum é descartada.

Quando o quantum termina e ainda resta CPU na rajada, conclusões de E/S e chegadas ocorridas no mesmo tick são primeiro inseridas por *PID* crescente. O processo preemptado recebe um novo *ready_since* e é então reinserido no fim da fila. Retornos de E/S também entram no fim da fila; retornos simultâneos são desempatados pelo menor *PID*. Se nenhum outro processo estiver pronto, o mesmo *PID* continua com um quantum renovado, sem preemptção nem troca de contexto.

C. Prioridade não preemptiva

Nesta política, valores numéricos menores representam prioridades mais altas, na faixa de 0 a 9. A fila de prontos é ordenada pela tupla (*prioridade*, *ready_since*, *PID*), de modo que processos com a mesma prioridade são escolhidos

primeiro pelo maior tempo contínuo de espera e depois pelo menor *PID*. A prioridade é estática durante toda a execução.

A política é estritamente não preemptiva: a chegada ou o retorno de E/S de um processo com prioridade mais alta não interrompe a rajada atualmente em execução. A ordenação por prioridade só é consultada quando a CPU é liberada por término ou bloqueio. Retornos simultâneos de E/S seguem o mesmo desempate determinístico, sem depender da ordem interna das estruturas de dados.

D. Algoritmo proposto: PDBH

O PDBH (Prioridade Dinâmica Baseada em Histórico) foi desenvolvido para mitigar o risco crônico de inanição (*starvation*) presente no algoritmo de Prioridade Estática não preemptiva, ao mesmo tempo em que recompensa processos que aguardam muito tempo na fila e penaliza aqueles que monopolizam a CPU. A abordagem baseia-se em conceitos de envelhecimento (*aging*) e nas penalidades aplicadas por Filas de Múltiplo Nível com Realimentação (MLFQ).

Diferente dos algoritmos clássicos, o PDBH recalcula a prioridade efetiva de cada processo dinamicamente a cada ponto de decisão (despacho). A regra matemática do projeto estabelece que menores valores numéricos representam maior prioridade. A pontuação dinâmica é calculada por:

$$Score = Prioridade_Estatica - Bonus_Espera + Penalidade_CPU$$

O bônus de espera é calculado por $\lfloor (Tempo_Atual - Ready_Since) / 10 \rfloor$, concedendo 1 ponto de melhora na prioridade a cada 10 ticks na fila. A penalidade é calculada por $\lfloor CPU_Consumida / 5 \rfloor$, piorando a prioridade em 1 ponto a cada 5 ticks processados historicamente. Em caso de empate na pontuação final, vence o processo com o menor tempo de entrada na fila (*ready_since*) e, persistindo o empate, o menor *PID*.

Limitações e Custos: Como a pontuação é recalculada dinamicamente, o PDBH possui complexidade de seleção $O(N)$, exigindo a varredura completa da fila de prontos. Ademais, por ser uma política não preemptiva, o algoritmo não interrompe rajadas em andamento. Observa-se também um viés estrutural desfavorável a processos *CPU-bound* de vida longa, uma vez que o histórico de CPU consumida nunca decai, enquanto o tempo de espera é reiniciado a cada retorno de E/S.

IV. METODOLOGIA

Para garantir a significância estatística da avaliação, o protocolo experimental foi desenhado com 1.600 execuções isoladas. Foram definidos quatro cenários de carga de trabalho: Equilibrado, I/O-Bound, CPU-Bound e Desbalanceado (onde 70% dos processos possuem alta prioridade).

Para cada um dos quatro cenários, os quatro algoritmos (FCFS, RR, Prioridade e PDBH) processaram 100 cargas idênticas. Cada carga de trabalho continha 1.000 processos gerados a partir de 100 *seeds* aleatórias distintas.

As métricas coletadas incluem o Tempo de Retorno (*Turn-around*) médio, o número total de Trocas de Contexto e o Índice

de Jain aplicado ao Slowdown, utilizado para quantificar a equidade (justiça) da distribuição do tempo de resposta. A análise foi conduzida através do cálculo das médias acompanhadas do Intervalo de Confiança de 95% (IC95%), permitindo observar a sobreposição estatística entre os desempenhos.

V. RESULTADOS

Esta seção apresenta os resultados obtidos nas simulações, consolidados a partir de 1.600 execuções. Para cada cenário, os dados são expressos pela média acompanhada do Intervalo de Confiança de 95% (IC95%).

A. Cenário: Equilibrado

No cenário equilibrado, os resultados para a métrica de Turnaround Médio (em ticks) foram:

- FCFS: 26.112,67 [IC95%: 25.948,63 – 26.276,72]
- Round Robin: 33.817,33 [IC95%: 33.624,15 – 34.010,51]
- Prioridade: 23.189,82 [IC95%: 23.083,25 – 23.296,40]
- PDBH (Proposto): 26.112,71 [IC95%: 25.949,17 – 26.276,25]

Para a métrica de Equidade (Índice de Jain do Slowdown), os valores observados foram:

- FCFS: 70,91% [IC95%: 70,53% – 71,28%]
- Round Robin: 81,21% [IC95%: 81,09% – 81,32%]
- Prioridade: 31,10% [IC95%: 30,86% – 31,34%]
- PDBH (Proposto): 70,99% [IC95%: 70,62% – 71,37%]

O volume de Trocas de Contexto acompanhou a natureza preemptiva das políticas. O Round Robin contabilizou 12.260 trocas [IC95%: 12.213,18 – 12.308,61], enquanto os demais algoritmos (FCFS, Prioridade e PDBH) apresentaram exatamente 3.504,90 trocas [IC95%: 3.491,50 – 3.518,29].

B. Cenário: I/O-Bound

No cenário de alta intensidade de Entrada/Saída (I/O-Bound), os resultados para a métrica de Turnaround Médio (em ticks) foram:

- FCFS: 32.268,65 [IC95%: 32.221,99 – 32.315,31]
- Round Robin: 35.607,17 [IC95%: 35.554,87 – 35.659,47]
- Prioridade: 20.120,75 [IC95%: 20.090,48 – 20.151,02]
- PDBH (Proposto): 32.268,25 [IC95%: 32.221,65 – 32.314,84]

Para a métrica de Equidade (Índice de Jain do Slowdown):

- FCFS: 97,66% [IC95%: 97,64% – 97,68%]
- Round Robin: 97,32% [IC95%: 97,30% – 97,35%]
- Prioridade: 72,38% [IC95%: 72,30% – 72,47%]
- PDBH (Proposto): 97,65% [IC95%: 97,63% – 97,67%]

O número de Trocas de Contexto acompanhou a fragmentação imposta pelo quantum no Round Robin, que contabilizou 10.212,15 trocas [IC95%: 10.200,51 – 10.223,79]. Os algoritmos não preemptivos registraram 6.498,87 trocas [IC95%: 6.492,07 – 6.505,67].

C. Cenário: CPU-Bound

Para o cenário CPU-Bound, caracterizado por rajadas longas de processamento e poucas requisições de E/S, os resultados para a métrica de Turnaround Médio (em ticks) foram:

- FCFS: 49.180,78 [IC95%: 48.949,41 – 49.412,14]
- Round Robin: 72.641,57 [IC95%: 72.367,75 – 72.915,38]
- Prioridade: 41.199,78 [IC95%: 41.054,63 – 41.344,94]
- PDBH (Proposto): 49.192,65 [IC95%: 48.959,46 – 49.425,84]

Para a métrica de Equidade (Índice de Jain do Slowdown):

- FCFS: 84,22% [IC95%: 84,06% – 84,39%]
- Round Robin: 96,29% [IC95%: 96,26% – 96,32%]
- Prioridade: 54,79% [IC95%: 54,56% – 55,02%]
- PDBH (Proposto): 84,26% [IC95%: 84,09% – 84,44%]

As Trocas de Contexto refletiram a retenção da CPU: o Round Robin atingiu 20.714,96 trocas [IC95%: 20.658,90 – 20.771,01], enquanto os algoritmos não preemptivos registraram 1.996,75 trocas [IC95%: 1.991,98 – 2.001,51].

D. Cenário: Desbalanceado (70% Alta Prioridade)

Neste cenário, onde a grande maioria dos processos chega com prioridade estática alta, os resultados para o Turnaround Médio (em ticks) foram:

- FCFS: 26.120,46 [IC95%: 25.969,06 – 26.271,86]
- Round Robin: 33.841,36 [IC95%: 33.663,37 – 34.019,35]
- Prioridade: 23.596,96 [IC95%: 23.491,27 – 23.702,64]
- PDBH (Proposto): 26.120,99 [IC95%: 25.970,05 – 26.271,92]

Para a métrica de Equidade (Índice de Jain do Slowdown):

- FCFS: 70,51% [IC95%: 70,14% – 70,88%]
- Round Robin: 81,09% [IC95%: 80,97% – 81,20%]
- Prioridade: 30,65% [IC95%: 30,37% – 30,93%]
- PDBH (Proposto): 70,57% [IC95%: 70,20% – 70,94%]

As Trocas de Contexto mantiveram o padrão dos cenários anteriores: 12.272,37 trocas para o Round Robin [IC95%: 12.228,45 – 12.316,28] e exatas 3.508,79 trocas para as políticas não preemptivas [IC95%: 3.496,66 – 3.520,91].

VI. DISCUSSÃO

A análise do cenário equilibrado evidencia um severo *trade-off* entre o tempo de entrega e a justiça do sistema. A política de Prioridade Estática obteve o menor Turnaround absoluto (23.189,82 ticks), isolando-se estatisticamente dos demais algoritmos. Contudo, esse baixo tempo médio é alcançado em detrimento dos processos menos prioritários, o que é atestado matematicamente pela queda drástica do Índice de Jain para 31,10%, configurando um estado grave de inanição (*starvation*).

O Round Robin, em contraste, obteve a maior equidade do grupo (Jain de 81,21%), mas penalizou o Turnaround (33.817,33 ticks). Essa degradação é explicada mecanisticamente pelo fatiamento do *quantum*, que multiplicou por três a incidência do custo unitário de troca de contexto (12.260 trocas contra 3.504 dos demais).

Por fim, os intervalos de confiança (IC95%) do algoritmo proposto (PDBH) e do FCFS sobrepueram-se integralmente em todas as métricas observadas no cenário equilibrado, indicando ausência de diferença estatisticamente significativa entre eles. Por se tratar de um ambiente homogêneo e de uma política não preemptiva, o mecanismo de pontuação dinâmica do PDBH não encontrou variações suficientes de histórico para sobrepor-se fortemente à ordem de chegada, operando, na prática, como um espelho estrutural do FCFS.

A análise do cenário I/O-Bound revela que a predominância de bloqueios por Entrada/Saída equaliza o desempenho das políticas não preemptivas voltadas à ordem de chegada. O algoritmo proposto (PDBH) e o FCFS registraram sobreposição integral de seus Intervalos de Confiança (IC95%) em todas as métricas, alcançando uma equidade quase perfeita (Índice de Jain superior a 97,6%).

Mecanicamente, essa similaridade estatística ocorre porque processos I/O-Bound abandonam a CPU frequentemente. Ao retornarem da operação de E/S, recebem um novo *ready_since*. Essa dinâmica de renovação constante impede que os processos acumulem o bônus de espera necessário na fórmula do PDBH para alterar a prioridade efetiva, anulando o diferencial do algoritmo.

A política de Prioridade Estática manteve o menor Turnaround absoluto (20.120,75 ticks), mas novamente ao custo da equidade, com o Índice de Jain caindo para 72,38% mesmo em um cenário onde a CPU fica livre com frequência. O Round Robin, por sua vez, demonstrou que o fatiamento de tempo é redundante para cargas I/O-Bound, pois os processos já liberam a CPU voluntariamente. A aplicação do *quantum* apenas inflou as Trocas de Contexto em relação aos demais (10.212 contra 6.498), resultando no pior Turnaround do cenário (35.607,17 ticks) e em um IC95% isolado negativamente.

No cenário CPU-Bound, a predominância de rajadas longas de processamento exacerbou as características de cada política. O Round Robin alcançou a equidade quase ideal (Jain de 96,29%), pois o fatiamento pelo *quantum* forçou o revezamento da CPU entre as tarefas longas. O custo mecânico dessa justiça foi o aumento exponencial nas Trocas de Contexto (20.714 contra 1.996 dos demais algoritmos), resultando em uma degradação severa do Turnaround (72.641,57 ticks, com IC95% isolado). Tarefas CPU-Bound raramente bloqueiam por E/S, obrigando o RR a exaurir o *quantum* e preemptar processos em quase todos os ciclos.

A política de Prioridade obteve o melhor Turnaround (41.199,78 ticks), mas registrou o menor Índice de Jain do experimento (54,79%). Essa discrepância evidencia que a retenção ininterrupta do processador por tarefas de alta prioridade impõe um atraso punitivo aos processos de menor prioridade da fila.

O algoritmo PDBH manteve a equivalência estatística com o FCFS, apresentando sobreposição completa de IC95% em Turnaround e Jain. Mecanicamente, essa limitação decorre da natureza não preemptiva do PDBH. Enquanto um processo CPU-Bound monopoliza o processador, todos os demais processos na fila de prontos acumulam bônus de espera simulta-

neamente. Quando a CPU finalmente é liberada, o processo que está há mais tempo na fila naturalmente possuirá a melhor pontuação, replicando a ordem do FCFS. A penalidade por consumo de CPU afeta as tarefas de forma homogênea neste cenário, não sendo suficiente para alterar a dinâmica de despacho.

O cenário desbalanceado expõe a fragilidade extrema da política de Prioridade Estática. Ao inundar a fila de prontos com 70% de processos de alta prioridade, a inanição atinge seu pico. A política alcançou o melhor Turnaround do cenário (23.596,96 ticks), porém obteve o pior Índice de Jain de todo o experimento (30,65%). Os 30% de processos com baixa prioridade foram severamente punidos, aguardando na fila até que quase toda a carga principal fosse exaurida.

O algoritmo PDBH, projetado justamente para mitigar a inanição através do envelhecimento, novamente não apresentou diferença estatisticamente significativa em relação ao FCFS, com sobreposição completa de seus Intervalos de Confiança (IC95%). Esse resultado evidencia uma limitação arquitetural central: a ausência de preempção. Em um cenário inundado de requisições, o bônus concedido por tempo de espera cresce de forma uníssona para toda a fila. Quando a CPU é finalmente liberada, o processo com maior tempo de chegada naturalmente possui o maior bônus acumulado, o que anula os pesos da fórmula e faz o PDBH degenerar para um comportamento espelhado da ordem de chegada (FCFS).

VII. CONCLUSÃO

A execução simulada de 1.600 cargas de trabalho permitiu avaliar os algoritmos de escalonamento sob um rigoroso protocolo estatístico. Os resultados demonstram que não existe um algoritmo universalmente superior, mas sim escolhas baseadas em *trade-offs* sistêmicos. A política de Prioridade não preemptiva maximiza a vazão e minimiza o *Turnaround*, mas falha criticamente na equidade (Jain $\approx$ 30% a 54%), gerando inanição. O Round Robin garante a justiça do processador (Jain superior a 80% em todos os cenários), mas o fatiamento do *quantum* infla o volume de trocas de contexto em até três vezes, degradando o tempo de entrega.

O algoritmo proposto (PDBH) demonstrou que o uso de pontuações dinâmicas de histórico em arquiteturas puramente não preemptivas é inócuo. A sobreposição constante de seus Intervalos de Confiança com o FCFS comprova que, sem o mecanismo de interrupção (preempção) para aplicar as penalidades de imediato, o peso do tempo de espera anula as variações da fila, revertendo as decisões à simples ordem de chegada.

REFERÊNCIAS

- [1] Silberschatz, A., Galvin, P. B., e Gagne, G. Sistemas Operacionais com Java. Elsevier, 2015.